NTU EMPLOYEE CLAIM SYSTEM · DEVELOPER BUILD PACK

ECS Pre-Trip Travel Request — Technical Build Specification

Audience: implementation / OutSystems build team **Status:** prototype-validated, build-ready **Generated:** 2026-10-22

> **Audience:** the implementation / OutSystems build team. **Purpose:** everything needed to build the module — data model, enumerations, configuration, the integration seam, screen specs, the full server-action surface, and every business-logic algorithm, at implementation level. **Companion documents:** the *Build Spec* (design decisions DD-01…DD-29, published artifact) and the *Feature Walkthrough* (end-user screens). This document is the implementation-level reference; where it and the design Build Spec differ on intent, the Build Spec's design decisions govern. **Reference prototype:** Next.js 16 (React 19 Server Components + Server Actions), Prisma 6, SQLite. The prototype is the executable specification — every rule below is implemented and verifiable in it.

0. How to read this document

- **§1** sets the architecture and the ECS platform boundary (what is reused vs built).
- **§2** is the **integration seam** — the exact ECS services and reference-data shapes the vendor must wire to live ECS/HR/DOA/Finance. Build this first; everything else depends on it.
- **§3–§5** are the **static build inputs**: the data model, the enumerations/state machines, and the runtime configuration + policy reference rates.
- **§6–§7** are the **surface**: screens and the server-action (API) catalogue.
- **§8–§10** are the **logic**: cost, entitlement, policy, validation, routing, settlement, duplicates, the TMC integration contract, and the claim engine — each with its algorithm.
- **§11–§13** cover lifecycle sequences, non-functional requirements, and the design-decision index.

Platform mapping (prototype → OutSystems): Screens/Blocks ← pages; Server Actions ← server actions; Entities ← Prisma models; Integration (REST / Service Actions) ← the mock ECS services in §2. Field types and rules are platform-agnostic.

1. Architecture & technology

1.1 Layers

LAYER	RESPONSIBILITY
Presentation	Wizard (Traveller → Trip → Estimates → Charging → Policy → Submit), approver/DOA queue, TE claim, admin/config, finance & reporting. Server-rendered with client interactivity for the itinerary builder, charging splitter and live validation.
Domain logic	Approval-route engine, class-entitlement derivation, cost/ODA/accommodation computation, policy evaluation, validation, charging roll-up, settlement, duplicate detection, claim workflow & auto-grant.
Integration seam	`EcsIdentity` · `EcsRoles` · `EcsDelegation` · `EcsReference` · `EcsFx` · `EcsPolicy` · `EcsTravelClassRegister` · `EcsCharging` · `EcsWorkflow` + the TMC adapter and notification/visa hooks. The one place wired to live systems (§2).
Transactional store	Requests, travellers, legs, estimates, allocations, policy checks, approval steps, bookings, segments, deviations, claims, claim lines, versions, audit, integration messages, attachments (§3).
Reference masters	Read-only ECS masters (identity, DOA lines, charging accounts, policy caps/ODA/class register, locations, FX, workflow bands, TMC providers). **Never written by this module.**

1.2 Module boundary — reuse vs build

- **Reused (read-only) from ECS/HR/DOA/Finance:** employees & appointments, reporting lines, DOA lines + amount bands, charging accounts (CC/WBS), policy caps & ODA rates, travel-class register, FX rates, locations/airport master, TMC providers, GST codes, GL accounts. The module **reads** these through the §2 service interface — it does not re-implement or maintain them.
- **New (transactional, owned by this module):** everything in §3.
- **New (maintained in this module):** the runtime Module Settings (§5), the Integration-Contract field/guard overrides (§9), and the Provost list (§3).

1.3 Build & run (prototype reference)

- `npm run dev` — dev server. `npm run seed` (`prisma db seed`) seeds the single `ModuleSettings` row from defaults (§5); demo scenarios are seeded lazily when the request table is empty.
- **Schema changes:** additive migrations only. `prisma db push` for the dev DB; generate a marked migration with `prisma migrate diff --from-migrations … --to-schema-datamodel … --script` + `prisma migrate resolve --applied <name>` to record an already-applied column without a destructive reset. Never `migrate dev` /reset against seeded data.
- **SQLite note:** SQLite has no native enums; all status/type columns are `String` whose allowed values are the enumerations in §4. On OutSystems use Static Entities or validated attributes.

2. Integration seam — ECS reference masters (build this first)

These are the **mock ECS core services**. In production each becomes a public server action of the existing ECS module, reading live ECS/HR/DOA/Finance. The module consumes them **read-only**; there is no CRUD or maintenance screen here (maintenance of masters stays in ECS).

2.1 Service interface

SERVICE	METHOD	RETURNS / PURPOSE
EcsIdentity	`entity()`	`{ id, name }` — the employing entity (NTU).
	`employee(id)`	`Employee` (§2.2) or undefined.
	`allEmployees()`	directory (used by approver pickers).
	`department(id)`	`Department` (§2.2).
	`allDepartments()`	
	`hasRole(id, role)`	boolean.
	`reportingOfficer(employeeId)`	the RO `Employee`, derived from `reportingOfficerId`.
EcsRoles	`employeesWithRole(role)`	all employees holding an ECS role (§4 `ROLE`).
EcsDelegation	`forDelegate(delegateId)`	delegations held by a delegate.
	`canCreateTravelRequestFor(delegateId)`	principal ids a delegate may raise for.
EcsReference	`expenseType(id)` / `expenseTypes()`	expense-type master (§2.2).
	`travelPurpose(id)` / `travelPurposes()`	purpose master (`isResearch` flag).
	`country(code)` / `city(code)` / `airport(code)` + list forms	location master (IATA).
	`travelClass(id)` / `travelClasses()`	class master (rank, costIndex).
	`currencies()` / `fxRates()`	currency + FX display.
	`flightHours(destCityCode)`	indicative SIN→dest flight hours (class derivation fallback; default 12).
	`highRiskForCountry(code)` / `highRiskDestinations()`	§4.8 advisory master.
	`rejectionReasons()`	send-back reason list.
EcsFx	`sgdPerUnit(currency, onDate?)`	effective-dated (month of `onDate`) rate.

SERVICE	METHOD	RETURNS / PURPOSE
	`toSgd(amount, currency, onDate?)`	conversion; re-locked at submission.
EcsPolicy	`hotelCap(cityCode)`	nightly cap: city-specific wins, else the city's **country** cap (§5.2).
	`odaRate(countryCode)` / `odaMonthlyRate(countryCode)`	daily + monthly-allowance ODA rate.
	`tolerances()`	deviation/amendment tolerances (§5.3).
	ODA tier constants	`odaTier1EndDay=15, odaTier1Pct=100, odaTier2EndDay=31, odaTier2Pct=75, odaMonthlyDays=30` (§8.1).
EcsTravelClassRegister	`entry(employeeId, onDate)`	effective register row (§2.2).
	`entitledForDuration(employeeId, hours, onDate)`	entitled class for one leg (Economy if below `minHours` or not in register).
	`entitledForItinerary(employeeId, legs, onDate)`	`{ classId (highest-wins), basis, perLeg[] }` across the itinerary (§8.2).
	`rank(classId)`	class rank (1=Economy … 3=Business).
EcsCharging	`code(code)`	`ChargingCode` (§2.2) or undefined.
	`codes()`	all CC/WBS.
EcsWorkflow	`bands()`	amount→DOA-tier bands (§5.3).
	`bandForAmount(amountSgd)`	the band whose range contains the amount → `doaTierIndex`.

2.2 Reference-data shapes

```
Employee { id; name; title; departmentId; reportingOfficerId: string|null; roles:
Role[];
           isTraveller; travelPolicyGroup; workerType?:
ACADEMIC|RESEARCH|ADMIN|EXECUTIVE;
           email?; businessArea?; defaultChargingCode?; travelEligible? }

Department { id; name; kind: SCHOOL|RESEARCH_INSTITUTE; headId;
             doaLine: string[];          // DOA escalation chain, index 0 = tier-1
(amount bands)
             researchDoaLine?: string[] } // research-route DOA chain

ChargingCode { code; name; type: CC|WBS; subType?: PROGRAMME|PROJECT; departmentId;
               companyCode; businessArea; isResearch; active?; validTo?;
               fundingOwnerId?; crossCharge? }

ExpenseType { id; teCode; name; category; glAccount; gstCode; taxRatePct;
              preTripApplicable; isMajorCost; estimateFormType:
AIRFARE|ACCOMMODATION|ODA|STANDARD;
              varianceThreshold }

TravelClass { id; name; rank: 1|2|3; costIndex }
TravelClassRegisterEntry { employeeId; displayName; entitledClassId; policyGroup;
                           minHours; effectiveFrom; effectiveTo }
WorkflowBand { id; minSgd; maxSgd: number|null; doaTierIndex; label }
HotelCap { cityCode; capNightlySgd; effectiveFrom }     CountryHotelCap { countryCode; …
}
OdaRate { countryCode; dailyRateSgd; monthlyRateSgd; effectiveFrom }
```

Critical wiring contracts:

- `Department.doaLine` is the authority chain; `bandForAmount(approvalAmount).doaTierIndex` indexes it. A research request uses `researchDoaLine`.
- `Employee.reportingOfficerId` is the sole source for the RO step (walked up the chain for self-approval escapes).
- `ChargingCode` carries `departmentId` (→ DOA owner), `businessArea` (→ cross-BA), `isResearch`, `active` / `validTo` (→ closed-account block), `crossCharge` + `fundingOwnerId` (→ funding-owner step).

---

3. Data model (entities)

SQLite strings stand in for enums (values in §4). All child entities cascade-delete from `TravelRequest` unless noted. IDs are `cuid()` unless noted.

3.1 `ModuleSettings` (single row, id `SETTINGS`)

Runtime configuration — see §5 for every field, type, default and effect.

3.2 `TravelRequest` (header + lifecycle)

FIELD	TYPE	DEFAULT	NOTES
id	String PK	cuid	
requestNumber	String **unique**		`TR-YYYY-NNNNNN`
authorisationNo	String? **unique**		`TA-YYYY-NNNNNN`, set on final approval
status	String	`Draft`	§4 request status
bookingStatus	String	`Not Sent`	§4 booking status (maintained separately)
isGroup	Boolean	false	
requestorId	String		creator (may be non-traveller)
travellerId	String		primary traveller (`GUEST` for a non-employee)
travellerType	String	`EMPLOYEE`	`EMPLOYEE | GUEST`
guestName / guestEmail / guestOrg	String?		guest identity (no HR record)
additionalApproverId	String?		traveller-selected extra approver
selectedDoaId	String?		traveller-selected DOA (not pre-selected)
provostId	String?		selected Provost for a guest class upgrade
isEmergency / isRetrospective	Boolean	false	emergency lane flags (DD-23)
emergencyJustification / emergencyAuthoriserId	String?		
emergencyRatified / emergencyOutsideWindow	Boolean	false	
entityId / departmentId	String?		
isResearch	Boolean	false	
purposeId / description	String?		trip header
tripType	String	`ROUND_TRIP`	`ROUND_TRIP | ONE_WAY | MULTI_CITY`

FIELD	TYPE	DEFAULT	NOTES
destCountry / destCity	String?		derived main destination (longest-stay leg)
startDate / endDate	DateTime?		official travel dates
personalStart / personalEnd	DateTime?		personal extension
personalAck	Boolean	false	personal-portion-at-own-cost ack
highRiskAck / highRiskApproverAck	Boolean	false	§4.8 acknowledgements
visaLetterRequired	Boolean	false	§4.13
visaLetterNotifiedAt	DateTime?		set by the daily visa batch
tmcProviderId	String?		preferred TMC provider (null = auto)
eventStartDate / eventEndDate	DateTime?		§10 event dates
invitationRef	String?		
travelClassId / entitledClassId / classBasis / classJustification	String?		derived + chosen class (individual)
bookingArrangement	String?	`AUTO`	`AUTO | TMC-<id> | SELF_BOOKED | HOST_ARRANGED | NO_BOOKING`
bookingMethod	String?		TMC channel: `TMC_ONLINE | AGENT_ASSISTED | OFFLINE` (only when TMC-arranged)
chargingMode	String	`REQUEST`	`REQUEST | LINE`
hotelSettlement	String	`INDIVIDUAL`	**DD-19** `INDIVIDUAL | CTA` (hotel payment channel; see §8.8)
approvalAmountSgd	Float?		amount-basis result for DOA tiering
bookingDeadline / authorisationExpiry	DateTime?		§13.3 windows
currentVersion	Int	0	
createdAt / updatedAt	DateTime	now / @updatedAt	

Children: `travellers, versions, legs, expenses, allocations, policyChecks, approvalSteps, bookings, deviations, teLinks, claims, messages, auditEvents, attachments`.

3.3 `TravelRequestTraveller` (group members)

`requestId→TR; employeeId` (or synthetic `G-…` for a guest member); `travellerType; guestName/Email/Org; confirmed; confirmedAt; sharePct(=100); isRequestor; entitledClassId; chosenClassId; classBasis; classJustification; ownStartDate; ownEndDate; legOverrideNote`.

- Per-traveller class (DD-05) and optional per-traveller sub-itinerary dates (§13.20).

3.4 `ItineraryLeg`

`requestId→TR; travellerId?` (null = shared group leg; set = sub-itinerary); `seq; originCode; destCode; departDate?; arriveDate?; transportMode(=AIR); durationHours?; departTime?; bookingRequired(=true); isPersonal(=false); nights(=0); travelClassId?; entitledClassId?; chosenClassId?`.

3.5 `EstimatedExpense` + `AccommodationEstimate` + `ODAEstimate`

`EstimatedExpense`: `requestId→TR; travellerId?; category (AIRFARE|ACCOMMODATION|ODA|CONFERENCE|OTHER); expenseTypeId; currency(=SGD); foreignAmount; sgdAmount; sponsorForeign; sponsorSgd; estimateBasis? (QUOTED|CAP|LOWER|RATE); expectedDate?; autoDerived(=false); isShared(=false); shareMap? (JSON {employeeId:pct}); chargingCode?; allocJson? (JSON [{code,pct,io}]=100%); notes?; originCode?; destCode?; proposedClassId?; fareCeiling?; handoffStatus?`. Has optional 1-1 `accommodation` and `oda`.

- `autoDerived` marks a policy-seeded line (per-stop hotel / per-city ODA) that is **re-derived** on a dates/itinerary change (§8.1); not a manual line.

`AccommodationEstimate` (1-1, `expenseId` unique): `city; legSeq?; checkIn?; checkOut?; nights; personalNights; quotedNightly; capNightly; budgetedNightly; capVariance; conferenceHotel; exceptionOutcome? (PASS|EXCEPTION)`.

`ODAEstimate` (1-1, `expenseId` unique): `country; city?; legSeq?; arrive?; depart?; eligibleDays; personalDays; dailyRate; ratePct(=100)`.

3.6 `ChargingAllocation`

`requestId→TR; companyCode?; businessArea?; chargingType (CC|WBS); chargingCode; isResearch; percent; amountSgd?; internalOrder?; isMain`. Per-request split; must total 100% (AC06). `amountSgd` converts to percent on save in AMOUNT mode.

3.7 `PolicyCheck`

`requestId→TR; code; label; outcome (PASS|WARNING|EXCEPTION|HARD_STOP); detail?; travellerId?; resolved`. Written by the policy engine (§8.3) on submit / re-run.

3.8 `ApprovalStep`

`requestId→TR; seq; roleType (RO|DOA|RESEARCH_DOA|EXCEPTION|FUNDING_OWNER|ADDITIONAL_APPROVER|PROVOST); approverId?; onBehalfOf?; status (Pending|Approved|Rejected|SentBack|Skipped); decidedAt?; comments?`.

3.9 `TravelBooking` + `BookingSegment` + `BookingDeviation`

`TravelBooking`: `requestId→TR; travellerId?` (group fan-out); `tmcProviderId?; pnr?; ticketNo?; channel? (TMC|SELF_BOOKED); fare?; taxes?; fees?; hotelRate?; status(=Booked)`. One TR → **N bookings/PNRs** (DD-17). `BookingSegment`: `bookingId→booking;`

`type (AIR|HOTEL); originCode?; destCode?; bookedClassId?; departDate?; arriveDate?; amount?`. `BookingDeviation`: `requestId→TR; field; approvedValue?; bookedValue?; material; action? (Reapproval|LogOnly|UpdateBooking)`.

3.10 TE claim entities

`TravelExpenseClaim`: `claimNumber unique; requestId?→TR; claimantId; status (Draft|Submitted); reportType; trsNotBooked; travelStart/End?; additionalApprover1/2?; lessCorpCard; lessPersonal; chargingMode (MAIN|CLAIM|ITEM); chargingMainCode?; chargingJson?`. Children `lines, approvalSteps`, 1-1 `link`. `ClaimApprovalStep`: `claimId; seq; roleType (VERIFIER|EXCEPTION|RO|DOA|RESEARCH_DOA); approverId?; status; autoGranted; decidedAt?; comments?`. `TEExpenseLine`: `claimId; category; expenseTypeId; treatment (LOCKED|EDITABLE|REFERENCE|EXCLUDED); approvedSgd; bookedSgd?; actualSgd; varianceSgd; receiptOk; transactionDate?; reason?; receiptType (LOCAL_GST|FOREIGN_NO_GST|NO_RECEIPT); currency; foreignAmount?; sponsorSgd; chargingCode?; allocJson?`. `TravelExpenseLink`: `requestId→TR; claimId unique→claim; travellerId?`.

3.11 Supporting entities

- `TravelRequestVersion`: immutable submitted/approved snapshot — `version; reason (SUBMISSION|REAPPROVAL); approved; snapshot (JSON); createdAt`.
- `IntegrationMessage`: `direction (OUTBOUND|INBOUND); kind (TMC_HANDOFF|TMC_RESPONSE); tmcProviderId?; payload (canonical JSON)`.
- `AuditEvent`: `requestId?; actorId?; onBehalfOf?; kind (CREATE|SUBMIT|APPROVE|REJECT|SENDBACK|AMEND|INTEGRATION|TE_LINK|STATUS|COMMENT); summary; createdAt`. Comments (§13.21) are `kind=COMMENT` in the same stream.
- `Attachment`: `docType (Quotation|Conference Invitation|Approval Evidence|Other); fileName; sizeKb; contentType?; content: Bytes?; uploadedById?`.
- `Provost`: `schoolId; approverId; name; title; active`; unique `(schoolId, approverId)`, indexed `schoolId`. The per-school guest-upgrade authority list.
- `IntegrationFieldSetting` (`key` PK; `includeTmc; teTreatment`) and `IntegrationGuardSetting` (`key` PK; `enabled; value`) — admin overrides over the §9 contract defaults.

---

4. Enumerations & state machines

4.1 Request status (`REQUEST_STATUS`) & transitions

`Draft, Submitted, Pending RO Approval, Pending Additional Approval, Pending Exception Approval, Pending Provost Approval, Pending DOA Approval, Approved, Amendment In Progress, Sent Back, Rejected(legacy), Withdrawn, Cancelled, Expired, Closed`.

Allowed transitions (invalid transitions must not be offered — AC24):

```
Draft              → Submitted, Withdrawn
Submitted          → PendingAdditional, PendingDOA, PendingException, PendingProvost
PendingRO          → PendingDOA, PendingException, PendingProvost, SentBack
PendingAdditional  → PendingException, PendingProvost, PendingDOA, SentBack
PendingException   → PendingProvost, PendingDOA, SentBack
PendingProvost     → PendingDOA, SentBack
PendingDOA         → Approved, SentBack
Approved           → AmendmentInProgress, Cancelled, Expired, Closed
AmendmentInProgress→ Approved, SentBack, Cancelled
SentBack           → Draft, Withdrawn
Expired            → PendingRO, PendingDOA, Cancelled
Rejected/Withdrawn/Cancelled/Closed → (terminal)
```

> **A Travel Request has no "Reject"** (DD-12): dissent = **Send Back** (→ Sent Back → Draft). `Rejected` is retained only for legacy compatibility and is not a transition target.

4.2 Booking status (`BOOKING_STATUS`)

`Not Sent, Sent to TMC, Received by TMC, Booking In Progress, Traveller Action Required, Partially Booked, Booked, Travel Completed, Booking Failed, Self-Booked, Cancellation Requested, Cancelled` . **In-flight set** (`TMC_INFLIGHT_STATUSES`): Received by TMC, Booking In Progress, Traveller Action Required, Partially Booked. An inbound response is only accepted while in flight (§9.4).

4.3 Claim status (`CLAIM_STATUS`)

`Draft, Pending Verification, Pending Exception Approval, Pending RO Approval, Pending DOA Approval, Approved, Sent Back` . First pending step → status via `claimStatusForRole` .

4.4 Other enums

- `POLICY_OUTCOME` : `PASS | WARNING | EXCEPTION | HARD_STOP` .
- `APPROVER_ROLE` : `RO, ADDITIONAL_APPROVER, DOA, RESEARCH_DOA, EXCEPTION, PROVOST, FUNDING_OWNER, VERIFIER` .
- `BOOKING_METHOD` : `TMC_ONLINE, AGENT_ASSISTED, OFFLINE, HOST_ARRANGED, SELF_BOOKED, NO_BOOKING` . **TMC channel sub-methods** offered when TMC-arranged: `TMC_ONLINE, AGENT_ASSISTED, OFFLINE` .
- `BOOKING_ARRANGEMENT` : `AUTO, SELF_BOOKED, HOST_ARRANGED, NO_BOOKING` (plus `TMC-<id>` for a specific provider). **TMC-arranged** ⇔ `AUTO` or `TMC-…` (the gate for CTA hotel settlement and automatic hand-off).
- `EXPENSE_CATEGORY` : `AIRFARE, ACCOMMODATION, ODA, CONFERENCE, OTHER` .
- `TE_TREATMENT` : `LOCKED, EDITABLE, REFERENCE, EXCLUDED` .
- `ROLE` (ECS personas): `TRAVELLER, TRAVEL_REQUESTOR, RO, DOA, RESEARCH_DOA, EXCEPTION_APPROVER, TRAVEL_ADMIN, SYSTEM_ADMIN, FINANCE_VIEWER` .

---

5. Configuration & policy reference rates

5.1 Module Settings (runtime-togglable — AC03)

KEY	TYPE	DEFAULT	EFFECT
sameRouteResearch	bool	false	Off → research-charged requests route to the **Research DOA** line.
expenseScope	enum	ALL	`MAJOR_ONLY | ALL` — which expense types are available.
hotelEstimateBasis	enum	LOWER	`QUOTED | CAP | LOWER` — accommodation budget basis (§8.1).
airfareTreatment	enum	IN_TE	`DIRECT_SAP | IN_TE | REFERENCE` — **how** a *central* airfare line posts; it **no longer decides reimbursability** (DD-19 — §8.8).
teLinkageMandatory	enum	CONDITIONAL	`YES | NO | CONDITIONAL` — must a claim select an approved TR.
approvalAmountBasis	enum	NET_NTU	`GROSS | NET_NTU | MAJOR_COST | ALLOCATION` — amount used for DOA tiering (§8.1).
groupTravelEnabled	bool	true	Enables one-request-many-travellers.
groupMaxTravellers	int	10	Max travellers in one group (TMC group-fare limit); create blocked above.
groupGuestMixAllowed	bool	**false**	CFO review: groups are employees-only; guests use a separate Guest request (DD-13).
exceptionApproverRequired	bool	true	On → a policy exception inserts an Exception Approver before DOA; off → surfaced to the DOA.
roRequirement	enum	INDIVIDUAL_ONLY	`ALWAYS | INDIVIDUAL_ONLY | NEVER` — RO decision point (DOA is always required).
teAutoGrantTolerancePct	int	10	TE claim DOA auto-grant tolerance (% over approved reimbursable).
teAutoGrantToleranceAbsSgd	int	500	…and the absolute SGD cap.
crossBaThresholdSgd	int	500	Non-primary BA share above this must concur (§8.5).

KEY	TYPE	DEFAULT	EFFECT
selfBookingEnabled	bool	true	Allows the Self-Booked method + TR selection at claim.
selfApprovalLimitSgd	int	0	Amount ≤ which a traveller-approver may self-approve (0 = never).
attachmentMaxMb	int	7	Max upload size per file.
chargingSplitMode	enum	PERCENT	`PERCENT | AMOUNT` entry mode.
declarationText / coiText	string	(default text)	Submit-step declaration + COI statements.
sftpConfig	string	(display)	TMC transport config (display only).
authorisationValidityDays	int	30	TA validity window (§13.3).
bookingDeadlineDays	int	14	Booking deadline window (§13.3).
emergencyLeadDays	int	14	DD-23 emergency standard lead-time window; beyond it the out-of-window justification + confirmation are required.

5.2 Reference rates (ECS policy masters — read-only)

Hotel caps — resolve a nightly cap by: city-specific cap first, else the city's **country** cap.

- City caps (SGD/night): TYO 350, OSA 300, LON 280, PAR 300, SIN 200.
- Country caps (fallback, SGD/night): AU 260, CN 240, DE 270, EG 180, FR 290, GB 300, HK 280, IN 180, JP 320, KR 260, MY 150, SG 200, US 330.

ODA rates — `dailyRateSgd` + `monthlyRateSgd` per country (sample): AU 125/2250, CN 100/1800, DE 120/2200, EG 90/1600, FR 110/2000, GB 130/2350, HK 115/2050, IN 85/1500, JP 120/2200, KR 110/2000, MY 75/1350, US 140/2500.

ODA duration tiers: days 1–15 → 100% of daily; days 16–31 → 75%; days 32+ → `monthlyRate ÷ 30` per day (§8.1).

5.3 Workflow bands, tolerances, class master

Workflow bands (amount → DOA tier):

BAND	RANGE (SGD)	DOATIERINDEX
BAND-A	0 – 5,000	0 (DOA-1)
BAND-B	5,000.01 – 20,000	1 (DOA-2)
BAND-C	> 20,000	2 (DOA-3)

Tolerances: booking fare `+10%` or `+SGD 200` (whichever lower); date shift `2 days`; estimate amendment `+10%` or `+SGD 500`.

Travel classes: `TC-ECO` Economy (rank 1), `TC-PEY` Premium Economy (rank 2), `TC-BIZ` Business (rank 3).

Policy thresholds (tuning): `shortLeadDays 14, longTripDays 14, highEstimateSgd 20000, severityHighSgd 2500, severityMediumSgd 500`.

---

6. Screens (UI build spec)

Wizard progress (stepper): **Traveller → Trip → Estimates → Charging → Policy Review → Submit**. Each step soft-saves and is navigable; mandatory-field and date validation are deferred to Submit so a partial step can be parked as a draft.

ROUTE	SCREEN	PURPOSE / KEY FIELDS / ACTIONS
`/dashboard`	Travel Request Dashboard	Traveller-scoped (admins see all); groups by drafts/sent-back, pending, approved-unbooked, completed-unclaimed; search/type/status filters; request-type flag S/E.
`/requests/new`	Create / Traveller	Pick traveller/group/guest; gated by remit (approver-only personas cannot create — §8 guards). Fields: traveller, requestor, entity/dept defaults.
`/requests/[id]`	Traveller step	Traveller & delegate/PA context.
`/requests/[id]/trip`	Trip details	Purpose; booking **arrangement** + TMC **method**; resolved TMC (read-only); description; invitation ref; visa-letter flag; **dates** (official start/end, event start/end, personal extension); **itinerary** (trip-type preset + always-on leg builder: origin/dest/depart/arrive/nights/class/personal). Upfront **duplicate/overlap banner** (§8.9). Action `saveTrip`.
`/requests/[id]/estimates`	Estimates	Auto-derived per-city **ODA** + per-stop **hotel** lines (§8.1); **hotel-settlement toggle** CTA/Individual (gated to TMC arrangements — §8.8); add Airfare / Accommodation / ODA / Other; inline hotel-rate edit (above-cap justification mandatory). Upfront duplicate banner.
`/requests/[id]/charging`	Charging	Request- or line-level split across CC/WBS (must total 100%); auto-allocate by traveller's Workday department; AMOUNT/PERCENT entry.
`/requests/[id]/policy`	Policy Review	Runs the policy engine (§8.3); lists PASS/WARNING/EXCEPTION/HARD_STOP; re-run. Upfront duplicate banner.
`/requests/[id]/review`	Review & Submit	One-page summary (Trip incl. **PNR provenance** once booked — §9.5, Charging, Estimates, route preview with DOA selector + Additional Approver); declaration + COI; high-risk ack; emergency-lane panel; **hard-stops + blocking duplicate disable Submit**. Action `submitRequest`.
`/requests/[id]/approve`	Approver / DOA	One-page Approval Summary (traveller, trip, cost & charging, exceptions emphasised, "You are approving" statement); Approve / Send Back (reason).
`/requests/[id]/booking`	Booking & TMC	Send to TMC (manual fallback), receive booking, set TMC status, mark self-booked, re-send, amend & re-route, cancel; per-PNR cards (PNR/ticket/fare/status + covers-route); ↻ **Refresh from TMC** (TA-keyed pull — §9.4); SAP airfare posting (DIRECT_SAP).
`/requests/[id]/pack`	Approval pack	Printable consolidated pack.

ROUTE	SCREEN	PURPOSE / KEY FIELDS / ACTIONS
`/requests/[id]/details`	Full details	Full detail cards behind the one-page summary.
`/claims/[id]`	TE claim	Pre-populated from the TR (§10); edit actuals; would-auto-grant indicator; submit → claim route.
`/approvals`	RO / DOA dashboard	Live Travel Requests / Expense Claims tabs, per-approver pending counters; exception rows shaded; Review → the summary.
`/workbench`	Travel Administration	Admin overrides, TMC ops.
`/finance`	Finance Oversight	Read-only committed-spend / pipeline / exceptions.
`/reports`	Reporting	Operational/financial/compliance aggregates.
`/visa-letters`	Visa Letters	Daily visa-letter batch (§8 — `runVisaLetterBatch`).
`/notifications`	Notifications	Approval/action/booking alerts.
`/integration`	Integration Contracts	Admin view/edit of the TMC/TE field + guard contract (§9), with a live payload preview.
`/config-viewer`	Configuration Viewer	Read-only reference masters.
`/settings`	Module Settings	System-Administrator-only; edits §5.1.
`/setup/provosts`	Provost List	Maintain the per-school guest-upgrade authority list.

7. Server actions (API surface)

All are server actions (mutations) returning void + `revalidatePath`, or `redirect` with a `?error=` query for a validation failure. Each checks edit/role guards. **Pre-trip** (`src/modules/pretrip/actions.ts`):

ACTION	SIGNATURE	EFFECT / GUARDS
createDraft	`(fd)`	Create an individual/group draft (remit-gated — §8).
createGuestDraft	`(fd)`	Create a guest (non-employee) draft.
createGroupDraft	`(fd)`	Create a group draft (employees-only unless `groupGuestMixAllowed`).
addGroupTraveller / removeGroupTraveller	`(id, …)`	Manage group members (≤ `groupMaxTravellers`).
saveTrip	`(id, fd)`	Persist trip header, dates, itinerary; **cross-field + leg validation** (§8.4); re-derives auto estimates on a dates/legs change (§8.1).
addLeg / removeLeg / moveLeg	`(id, …)`	Itinerary edits.
addAirfare / addAccommodation / addOda / addOther	`(id, fd)`	Add estimate lines (entitlement-scoped: no ODA for guests).
syncAutoEstimates	`(id, resetRates)`	Re-derive per-city ODA + per-stop hotel (§8.1). Internal, called by saveTrip.
setHotelSettlement	`(id, fd)`	**DD-19** set CTA/Individual; CTA accepted only for a TMC arrangement (§8.8).
updateAccommodationRate	`(id, expenseId, fd)`	Inline hotel nightly-rate edit (captures above-cap justification).
deleteExpense	`(id, expenseId)`	Remove a manual estimate line.
saveCharging	`(id, fd)`	Persist allocation split (request/line); must total 100%.
runPolicy	`(id)`	Re-run the policy engine; persist `PolicyCheck` rows.
submitRequest	`(id, fd?)`	**Final submit gate** — all hard-stops resolved, no blocking duplicate, destination present, DOA selected, emergency-lane + high-risk + guest guards (§8.3/§8.4/§8.9/§8.10); build route; set Pending-* status.
approveStep	`(id, fd?)`	Approve the current pending step; advance or finalise (issue TA).
sendBack	`(id, fd)`	Send back with a reason → Sent Back.
withdrawRequest / reopenDraft	`(id)`	Draft/sent-back lifecycle.

ACTION	SIGNATURE	EFFECT / GUARDS
cancelRequest / receiveCancellation	`(id, fd?)`	§34 cancellation (local before hand-off, TMC-mediated after).
handoffToTmc	`(id)`	Transmit the governed booking instruction (TMC arrangements only).
receiveBooking	`(id, fd)`	Ingest the inbound booking(s) — group fan-out + multi-PNR split (§9.4); captures segments incl. arrive dates; booking-deviation check.
refreshBookingsFromTmc	`(id)`	**TA-keyed pull** — re-fetch bookings for the authorisation; idempotent (§9.4).
setBookingTmcStatus	`(id, fd)`	Advance the TMC lifecycle / mark Travel Completed.
markSelfBooked	`(id)`	Record a self-booked trip.
postAirfareToSap	`(id)`	DIRECT_SAP airfare posting (when configured + booked).
resendToTmc / amendAfterFailedBooking	`(id)`	§6.4 failed-booking recovery.
runVisaLetterBatch	`()`	§4.13 daily visa-letter notification.
addComment / addAttachment	`(id, fd)`	Comments feed + evidence upload (≤ `attachmentMaxMb`).
copyToNewRequest	`(id)`	Clone to a new draft.

TE claim (`src/modules/te/actions.ts`): `createClaimFromRequest(requestId)` , `updateClaimActuals(claimId, fd)` , `submitClaim(claimId)` , `approveClaimStep(claimId)` , `sendBackClaim(claimId, fd)` , `updateClaimHeader(claimId, fd)` , `addClaimExpense(claimId, fd)` , `removeClaimExpense(claimId, lineId)` , `saveClaimCharging(claimId, fd)` .

8. Business logic / algorithms

8.1 Cost computation

Summary (`computeSummary(expenses, approvalAmountBasis)`): sums `gross` , `sponsorship` ; splits `major` / `incidental` by `expenseType.isMajorCost` ; `bookingCeiling` = Σ AIRFARE + ACCOMMODATION; `ntuFunded = gross − sponsorship` . **approvalAmount** = `GROSS` →gross, `MAJOR_COST` →major, `NET_NTU` / `ALLOCATION` /default→ntuFunded. `approvalAmount` drives DOA tiering (§8.5) and is **always the full committed spend** (unaffected by settlement — §8.8).

Accommodation (`computeAccommodation`): `capNightly = EcsPolicy.hotelCap(city)` ; `chargeableNights = max(nights − personalNights, 0)` ; `budgetedNightly` = `QUOTED` →quoted, `CAP` →cap, `LOWER` /default→ `min(quoted, cap || quoted)` ; `overCap = cap>0 && quoted>cap` ; `capVariance = overCap ? (quoted−cap)×chargeableNights : 0` ; `sgdAmount = budgetedNightly × chargeableNights` ; `outcome = overCap ? EXCEPTION : PASS` .

ODA — legacy day-split (`computeOda` , retained for reference): departure+return days at 50%, full days at 100%, personal days excluded.

ODA — duration-tiered (`computeOdaTiered({days, dailyRate, monthlyRate})` , the live per-city calc):

```
t1 = min(days, 15)                    amt = t1 × dailyRate × 100%
t2 = max(0, min(days,31) − 15)        amt = t2 × dailyRate × 75%
t3 = max(0, days − 31)                amt = t3 × (monthlyRate ÷ 30)
sgdAmount = round2(t1Amt + t2Amt + t3Amt)
```

Auto-derivation (`syncAutoEstimates(id, resetRates)`): from the leg dates it computes per-city stops — `from = leg.arriveDate ?? leg.departDate` , `to = nextLeg.departDate ?? nextLeg.arriveDate ?? req.endDate` , `nights = round((to−from)/day)` — and creates **one hotel line per stop** (city cap × nights; nightly rate reset to cap only when `resetRates`) and **one ODA line per city** (tiered, `days = nights`), keyed by `(travellerId, legSeq)` , reconciling (update/create/delete). Called by `saveTrip` with `resetRates = (dates/legs signature changed)` — a dates change supersedes a prior manual rate edit.

8.2 Travel class entitlement (§13.19)

Per flown leg, `entitledForDuration(employee, leg.durationHours, onDate)` = `register.entitledClassId` if `hours ≥ register.minHours` , else Economy (also Economy if not in the register). Across the itinerary the **highest-ranked** per-leg entitlement wins (critical for multi-city). Stored on the request (and per traveller / per leg). A **chosen** class above entitlement: (a) requires a justification — missing ⇒ hard-stop `CLASS_JUSTIFY` ; (b) raises the `CLASS` exception → Exception Approver (or DOA if the step is off). A guest defaults to Economy; a higher guest class additionally requires a selected Provost (`CLASS_PROVOST`) → Provost step (DD-25).

8.3 Policy engine (`evaluatePolicies(req, approvalAmountBasis)` → `PolicyCheck[]`)

Each rule is gated on `ruleActive(code, onDate)` (data-driven catalogue; `mandatory` rules always run). Worst outcome gates submission (`hasHardStop` blocks; `hasException` routes).

CODE	CATEGORY	OUTCOME	TRIGGER
DATES	Dates	HARD_STOP*	Start/end missing, or end < start.
DATE_PAST	Dates	HARD_STOP	Official **start date in the past** (start-of-today cutoff; same-day OK; drafts exempt).
LEG_OVERLAP	Dates	HARD_STOP*	A leg departs a city **before the previous leg arrives** (only when the prior arrive date is set).
ROUTE_GAP	Dates	WARNING	A leg's origin ≠ the previous leg's destination (possible missing connection).
ALLOC	Funding	HARD_STOP*	No allocation, or total ≠ 100%.
FUNDING	Funding	HARD_STOP*	Unknown charging code.
FUNDING_INACTIVE	Funding	HARD_STOP	A closed/inactive CC/WBS.
ESTIMATE	Estimate	HARD_STOP*	No estimate line.
HOTEL_CAP	Accommodation	EXCEPTION	Nightly rate above the city/country cap.
HOTEL_JUSTIFY	Accommodation	HARD_STOP	Above-cap hotel with **empty justification** (raised inside the HOTEL_CAP loop).
CLASS	Travel class	EXCEPTION	Chosen class above the derived entitlement.
CLASS_JUSTIFY	Travel class	HARD_STOP*	Class upgrade with no justification.
CLASS_PROVOST	Travel class	HARD_STOP*	Guest above Economy with no Provost selected.
OFFLINE	Booking	EXCEPTION	Offline booking (needs supporting quotation + approval).
LEAD	Lead time	WARNING	Departure within `shortLeadDays` (14).
LONG	Duration	WARNING	Trip > `longTripDays` (14).
HIGH	Estimate	WARNING	Approval amount > `highEstimateSgd` (20,000).
DUPLICATE	Duplicate	WARNING	Overlapping live trip, different destination (§8.9).
HIGH_RISK	High-risk	WARNING	High-risk destination advisory (+ mandatory traveller/approver ack).

`*` = `mandatory` (always enforced).

8.4 Validation rules (field / cross-field / submission)

Trip-step (`saveTrip`, blocks non-draft save): travel purpose, ≥1 leg with a destination, travel dates, booking arrangement **required**; `end ≥ start`; **official start not in the past** (same-day

OK); event dates within the official window; personal extension `end ≥ start` and **not overlapping** the official dates (before start or after end only); per leg `arrive ≥ depart`; each leg departs within the window (widened by a personal extension); legs **chronological** (`leg n+1 depart ≥ leg n depart`); **no overlap** (`leg n+1 depart ≥ leg n arrive`, only when the prior arrive is set). **Date gaps between legs are legitimate** (the city stay that drives hotel/ODA) and are **not** validated. **Submission (`submitRequest`):** all hard-stops resolved (AC06); no **blocking duplicate** (§8.9); a destination is present; a DOA is selected when candidates exist (DD-02); high-risk ack when applicable; emergency-lane gates (§8.10); no guest-attributed ODA/shared line; status transition in the allowed set; self-approval prevented (§8.6). **Claim:** no expense type absent from the approved request; a `NO_RECEIPT > 0` line raises a new exception; claim over the approved **reimbursable** amount beyond tolerance raises a new exception (§10).

8.5 Approval routing (`buildRoute(req, opts)` → ordered `RouteStep[]`)

Route order (each step **skips/escalates past** anyone in the self-approval set — §8.6):

1. **Additional Approver** — if the requestor selected one (and not self).
2. **RO** — per `roRequirement` (`ALWAYS`, or `INDIVIDUAL_ONLY` for non-group; `NEVER`=none). Resolved as the traveller's RO; walked up the reporting line if the RO is excluded.
3. **Funding Owner** — a cross-charge account with `crossCharge && fundingOwnerId`.
4. **Cross-BA concurrence** (§18) — when charging spans > 1 business area, every non-primary BA whose `approvalAmount × share%` exceeds `crossBaThresholdSgd` adds a concurring approver (its funding owner, else its DOA line); the primary-BA DOA still owns the request.
5. **Exception Approver** — when `hasException && exceptionApproverRequired`.
6. **Provost** — a guest travel-class upgrade with a selected Provost (DD-25).
7. **DOA** (always) — line = research line if `isResearch && !sameRouteResearch` else `doaLine`; tier = `bandForAmount(approvalAmount).doaTierIndex`; uses `selectedDoaId` when set (and not excluded), else `pickDoa` walks the line from the tier, escalating past excluded holders. Role = `RESEARCH_DOA` or `DOA`. `statusForStep(firstPending)` sets the Pending-* status. **Emergency** builds the *same* route (no step skipped) and annotates each step "retrospective ratification (book-first, DD-23)".

8.6 Self-approval prevention (DD-28 / §13.5)

`selfApprovalSet(req)` = every traveller + the requestor. No one in this set may be an approver: single-holder steps escalate up the relevant line (RO → reporting line; DOA → DOA tier line); multi-candidate steps pick the first eligible. Approver-only personas (DOA/Exception/Provost/SysAdmin/Finance) cannot **originate** a request at all (`guards.canCreateTravelRequest`).

8.7 Charging allocation & roll-up

`chargingMode` = `REQUEST` (one split for the whole request) or `LINE` (per-expense-line split via `EstimatedExpense.chargingCode` / `allocJson`). The request-level allocation driving routing is the **roll-up** of `net × %` over every line × split. "Auto-allocate by traveller's Workday department" fills each line from the attributed traveller's `defaultChargingCode`. Highest-charging department (`highestChargingDept`) = the department bearing the largest share (tie-break to the requestor's department) → the DOA owner.

8.8 Settlement channel (DD-19) & the reimbursable rule

Settlement decides **who fronts the money**, and therefore whether a claim line is **reimbursable** or **reference-only**. It is separate from authorisation and cost ownership: the approval amount still covers the full committed spend, and the CC/WBS allocation is unchanged. Single source of truth — `lineSettlement(req, category)`:

```
AIRFARE        → CENTRAL if isTmcArrangement(bookingArrangement) else EMPLOYEE  (self-booked flight = reimbursable)
ACCOMMODATION  → CENTRAL if hotelSettlement === 'CTA' (and TMC arrangement) else EMPLOYEE
ODA / OTHER    → EMPLOYEE (always)
```

`isReimbursable = (settlement === EMPLOYEE)`. **Reads:** (a) TMC payload — each hotel segment carries `billTo = CTA | TRAVELLER`; (b) claim treatment — CENTRAL lines (TMC airfare, CTA hotel) → `REFERENCE` (shown, not reimbursed), EMPLOYEE → `EDITABLE`; (c) DD-08 tolerance — compares the **reimbursable** (EMPLOYEE) portion on **both** the claim total and the approved baseline. `airfareTreatment` only governs *how* a central airfare line posts (DIRECT_SAP vs shown-in-claim), never reimbursability. The hotel toggle applies to individual + group alike, CTA offered only for a TMC arrangement.

8.9 Duplicate / overlap detection (DD-14)

`findDuplicateRequests(req)` — among **live** requests (Submitted · Pending* · Approved · Amendment In Progress) other than this one, that **share an employee traveller** (guests excluded) and whose **official dates overlap**: `sameDestination = (same country) && (same city when both set)`. **True duplicate** (same destination) ⇒ hard block at submit; **date clash** (different destination) ⇒ warning. **Surfaced upfront** on every editable step (Trip · Estimates · Charging · Policy) as soon as dates are known; the submit-stage hard block stays the backstop.

8.10 Emergency lane (DD-23)

Book-first/raise-later, bounded & audited. Gates on submit: a named **authorising officer** (not a traveller) + the authorising **e-mail attached** (mandatory evidence); **agent-assisted / offline TMC booking only**; destination mandatory; policy hard-stops still apply; **soft-gated by** `emergencyLeadDays` — within the window it is freely available, outside it a detailed justification (≥ min length) + an explicit out-of-window confirmation are required and the request is flagged *Outside standard window*. The **full approval route still runs** (same approvers, ratified retrospectively — §8.5); on submit the booking instruction is auto-handed to the TMC (DD-26) with an `EMERGENCY_BOOK_FIRST` indicator.

---

9. TMC / integration contracts

The **Integration Contract** (`src/config/integrationContracts.ts`) is the declarative source of truth for two boundaries: *Send to TMC* (outbound) and *Sync to TE* (claim). Admin overrides live in `IntegrationFieldSetting` / `IntegrationGuardSetting` and are merged at read time.

9.1 Outbound payload (Send to TMC) — `CONTRACT_FIELDS` (tmc)

Core (always sent): `authorisationNumber, travellerRef, bookingMethod`. Toggleable (default on): all group `travellers`, `bookingDeadline`, `costCeiling` (Σ airfare+accommodation), `approvedExceptions`, `tripPurpose`, `eventDates`, `approvalValidity`, `airSegments` (origin/dest/**depart+arrive date**), `airApprovedClass` (ceiling cabin), **`hotelSegments`** **(city/nights/cap +** **`billTo`** **CTA|TRAVELLER)**, `charging` (CC/WBS/BA for direct ERP posting), `approvalMetadata`, `travellerCategory` +contact, `personalIndicator`, `guestProfile` (PDPA — TMC secure link, guest-only). Emergency hand-off additionally carries the `EMERGENCY_BOOK_FIRST` indicator.

9.2 Message guards — `CONTRACT_GUARDS`

Outbound (all must hold to send): status = Approved; bookingStatus = Not Sent; bookingMethod ∈ TMC-served; a TA number is set. **Inbound** (to accept a response): a booking is in flight (in-flight set); a valid TA. **TE sync**: status ∈ {Approved, Closed}; a TA is set.

9.3 Group fan-out & multi-PNR split (DD-17)

A group hand-off emits **one instruction per traveller**, each with that traveller's effective itinerary/class/nights/cost share. A multi-city itinerary is commonly **split across PNRs** for competitive fares — on inbound a `≥3` -air-segment multi-city trip splits into PNR-A (first half + hotel) and PNR-B. Each PNR is its own `TravelBooking` ; the claim aggregates across PNRs. One TR → one claim → **N PNRs**.

9.4 Inbound — **TA-keyed retrieval** (DD-17)

The claimant creates the claim **from the TR**; the TMC exposes booking retrieval **by the Travel Authorisation number**, returning the **full set of PNRs under that TA** (not a PNR-by-PNR lookup). `bookingsForAuthorisation(req)` → `{ authorisationNo, pnrCount, bookings[{ pnr, ticketNo, providerId, travellerId, status, fareSgd, taxesSgd, feesSgd, hotelRateSgd, totalSgd, segments[{type,origin,dest,departDate,arriveDate,bookedClassId,amount}] }], totalFareSgd }` . **Push-primary** (`receiveBooking` stores bookings against the TR as they occur, so the claim pre-fills with zero PNR entry) with a **pull-by-TA refresh** fallback (`refreshBookingsFromTmc` , idempotent — an already-received booking is never duplicated). A self-booked trip returns an empty set (manual actuals). **Booking-deviation check:** booked aggregate airfare vs approved + tolerance (`+10%` or `+SGD 200` , lower) → material deviation routes to re-approval.

9.5 PNR provenance on the TR

`legPnrMap(req)` maps each shared AIR leg to the PNR that ticketed it (match origin+dest, and depart date when both present); an unmatched leg is left untagged (degrades to the TR-level summary rather than mis-tagging). The Review page shows per-leg PNR tags **and** a TR-level "Booking reference(s) · N PNRs" summary once booked.

---

10. TE claim engine

Create-from-TR (`createClaimFromRequest`): gated by the TE_SYNC guards; reuses an open draft; nets prior claims per category (AC17). `prepopulateClaim(req, claimantId, airfareTreatment, contract?, priorByCat?)` builds header + lines:

- **Header** fields (LOCKED mostly): traveller, entity, department, TR no, TA no, purpose, description, destination, dates, approved class (REFERENCE), charging (LOCKED), booking ref (REFERENCE).
- **Line treatment** — driven by settlement (§8.8): a line is **REFERENCE** when `!isReimbursable(req, category)` (TMC airfare, CTA hotel), else **EDITABLE**; an admin contract override still wins; EXCLUDED lines are dropped.
- Group: filtered to the claimant's attributed/shared lines; shared accommodation costed on the claimant's own nights.

Claim workflow (`buildClaimRoute`): Verifier → (Exception if a new exception) → (RO per `roRequirement`) → DOA (always; from the claim's **effective** charging department + amount band). **DD-08 auto-grant** (`doaAutoGrant`): the DOA step is auto-granted from the approved TR when **all** hold — TR is Approved/Closed with a TA; `claimTotal − approvedAmount ≤ tolerance` (% and SGD); no new expense type; no new exception; **unchanged charging DOA**. A changed

charging account routes to the **new** account's DOA (never auto-granted). Scope: `claimTotal` = Σ **non-REFERENCE** claim lines – corp-card – personal; `approvedAmount` = Σ **reimbursable** (EMPLOYEE-settled) approved expenses net of sponsorship — **like-for-like** so central airfare + CTA hotel are excluded from both sides.

---

11. Lifecycle sequences

Happy path: Create → Trip → Estimates → Charging → Policy → Review (pick DOA, declare) → Submit → route (Additional? ▸ RO? ▸ Funding/Cross-BA? ▸ Exception? ▸ Provost? ▸ **DOA**) → **Approved + TA issued** → auto hand-off to TMC (DD-26) → Booked → Travel Completed → **TE claim** (pre-filled; DOA auto-granted within tolerance). **Amendment:** a material change to an Approved request → Amendment In Progress → re-derive route → re-approve → revised TA. **Failed booking (§6.4 / DD-15):** TMC reports status (not an approval); not issued within the window → Booking Failed → re-send / amend & re-route (single-TA) / cancel & re-raise (frees the overlap under DD-14). A fare over tolerance routes to re-approval. **Cancellation (DD-22):** before hand-off → local withdraw/cancel; after transmit/book → via the TMC (`cancelRequest` → Cancellation Requested → `receiveCancellation`). **Emergency (DD-23):** submit with authoriser + e-mail → auto hand-off (`EMERGENCY_BOOK_FIRST`) → book first → full route ratified retrospectively.

---

12. Non-functional & build considerations

- **Authorisation & personas:** every screen/action is role-gated (§4 `ROLE`). Approver-only personas cannot create requests; creation is remit-gated (traveller/delegate/PA/Travel Admin). Delegation via `EcsDelegation` .
- **Self-approval:** enforced in routing (§8.6), not just the UI.
- **Audit:** every field/workflow/integration event and every comment is an `AuditEvent` (one chronological stream). Submitted/approved states snapshot to `TravelRequestVersion` .
- **PDPA (DD-21):** ECS stores only guest name/email/organisation; passport/profile is collected by the TMC via a secure link (the `guestProfile` payload field). Never store guest travel documents.
- **Attachments:** metadata + bytes in `Attachment` ; enforce `attachmentMaxMb` ; evidence is mandatory for the emergency authoriser's e-mail and an above-cap/offline justification is mandatory text.
- **Idempotency:** TMC hand-off and inbound ingestion are idempotent (a booking already sent/in-flight/booked is never re-sent or duplicated) so a re-submit/re-approval cannot double-book.
- **FX:** entry-date conversion, re-locked at submission (`EcsFx` with the submission month).
- **Integrity to confirm at discovery:** live integration contracts (TMC/OBT, HR/Workday, DOA, SAP Finance), non-functional targets & volumes, and the open decisions below.

---

13. Design-decision index

The design rationale for the behaviours above is catalogued as **DD-01…DD-29** in the companion *Build Spec* (design decisions section). Key ones referenced here: DD-01 (config-driven routing), DD-02 (DOA always/traveller-selected), DD-04/05 (class entitlement; group per-traveller), DD-07 (per-line charging), DD-08 (claim auto-grant), DD-13 (groups employees-only), **DD-14** (one

live authorisation; duplicate guard surfaced upfront), DD-17 (1 TR → N PNRs; **TA-keyed inbound + PNR provenance on the TR**), **DD-19** (settlement channel; reimbursable = employee-settled), DD-23 (emergency lane), DD-24 (dashboard approvals + one-page summary), DD-25 (guest upgrade → Provost), DD-26 (automatic TMC hand-off), DD-27 (destination mandatory), DD-28 (separation of duties).

Open decisions (confirm at detailed design): O-1 (claim auto-grant tolerance figures), O-4 (CTA scope beyond hotel/airfare — largely resolved by DD-19), O-5 (upgrade base-vs-differential split percentages, DD-20).

This specification describes the intended build. Final design may change subject to a formal discovery / detailed-design phase with the implementation vendor; material changes are tracked by version.

Implementation-level build specification derived from the reference prototype. Final design may change at a formal discovery / detailed-design phase; material changes are tracked by version.